



Mã hoá và Thám mã

Privacy Preserving Machine Learning with Fully Homomorphic Encryption

GVHD: TS. Võ Văn Khang

HVTH: Nguyễn Ngọc Đĩnh – 2441862007

Hà Anh Dũng – 2441862005

Nguyễn Minh Trung Nghĩa – 2441861021

Aug2025

Agenda

- Giới thiệu
- Cơ sở lý thuyết
- Tổng quan nghiên cứu
- Phương pháp thực hiện
- Thực nghiệm
- Kết luận & hướng phát triển
- Nguồn tham khảo



Agenda

- **Giới thiệu**
 - **Telling stories**
- Cơ sở lý thuyết
- Tổng quan nghiên cứu
- Phương pháp thực hiện
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Confidential Token Transfer

Meet Alice and Bob

Both users have encrypted balances on the blockchain. Click "Reveal" to see their actual ETH amounts!



Alice's Wallet

Encrypted Balance:

0x8f3a9c2b5e7d4f6a8c9e2b5f7d3a
6c8e9f2b5d7a3c6e9f2b5d7a3c6e9f
2b5d7a

Actual Balance:

10 ETH

Reveal Balance



Bob's Wallet

Encrypted Balance:

0x2d5f8a3c9e7b4f6d8a3c9e7b5f8d
3a6c9e7b5f8d3a6c9e7b5f8d3a6c9e
7b5f8d

Actual Balance:

???

Reveal Balance

Agenda

- **Giới thiệu**
 - ✓ Telling stories
 - **Giới thiệu & mục tiêu**
- Cơ sở lý thuyết
- Tổng quan nghiên cứu
- Phương pháp thực hiện
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Giới thiệu & Mục tiêu

Lý do chọn đề tài?

- Machine Learning (ML) hiện đại ngày càng **phụ thuộc vào dữ liệu lớn**, và **yêu cầu quyền riêng tư cao**, đặc biệt trong các lĩnh vực nhạy cảm (y tế, tài chính, quốc phòng).
- Nếu không bảo vệ quyền riêng tư:
 - Người dùng/tổ chức sẽ không chia sẻ dữ liệu.
 - Nguy cơ vi phạm luật (GDPR, HIPAA).
 - Rủi ro rò rỉ dữ liệu → mất niềm tin, thiệt hại uy tín/tài chính.
- Nhu cầu: "**Có thể huấn luyện/dự đoán mà không tiết lộ dữ liệu gốc?**"



Mục tiêu

- Tìm hiểu lý thuyết và áp dụng Fully Homomorphic Encryption (FHE) trong ML.
- Phân tích thách thức và giải pháp khi tích hợp FHE vào ML.
- Tập trung inference trên dữ liệu mã hoá bằng FHE.
- Thực nghiệm demo với Concrete-ML.

Agenda

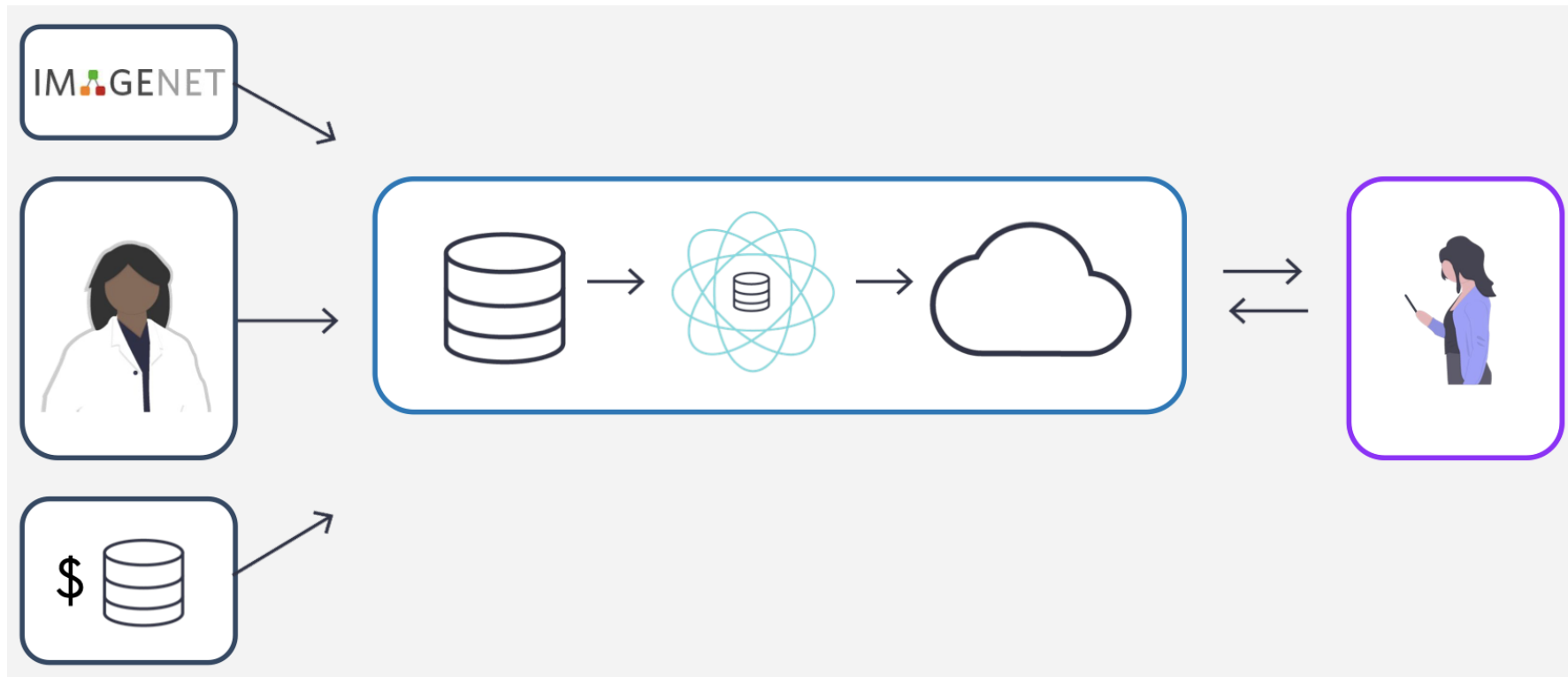
- **Giới thiệu**
 - ✓ Telling stories
 - ✓ Giới thiệu & mục tiêu
 - **ML/DL Workflow & Rủi ro**
- Cơ sở lý thuyết
- Tổng quan nghiên cứu
- Phương pháp thực hiện
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



ML Workflow & Rủi ro bảo mật

ML/DL Workflow truyền thống

Dữ liệu nhạy cảm **được gửi lên server/cloud** → dễ bị truy cập trái phép hoặc sử dụng sai mục đích.

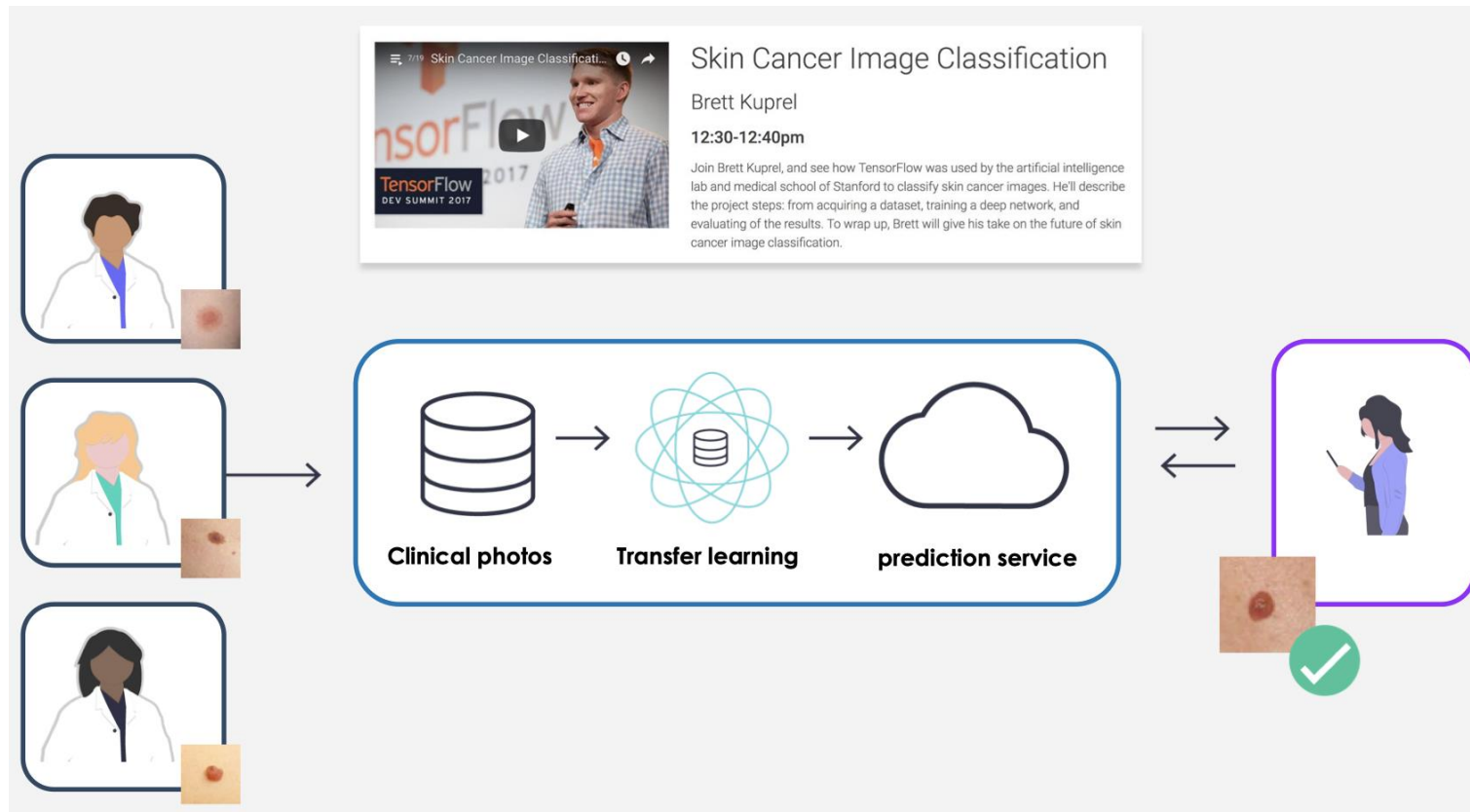


ML Workflow & Rủi ro bảo mật

Một ví dụ thực tế về ML/DL Workflow truyền thống

Phân loại ảnh ung thư da bằng học máy - bệnh viện gửi ảnh bệnh nhân lên **server AI không đáng tin cậy**.

Ảnh bệnh nhân **chứa thông tin nhạy cảm** → nguy cơ vi phạm luật bảo mật (HIPAA, GDPR).

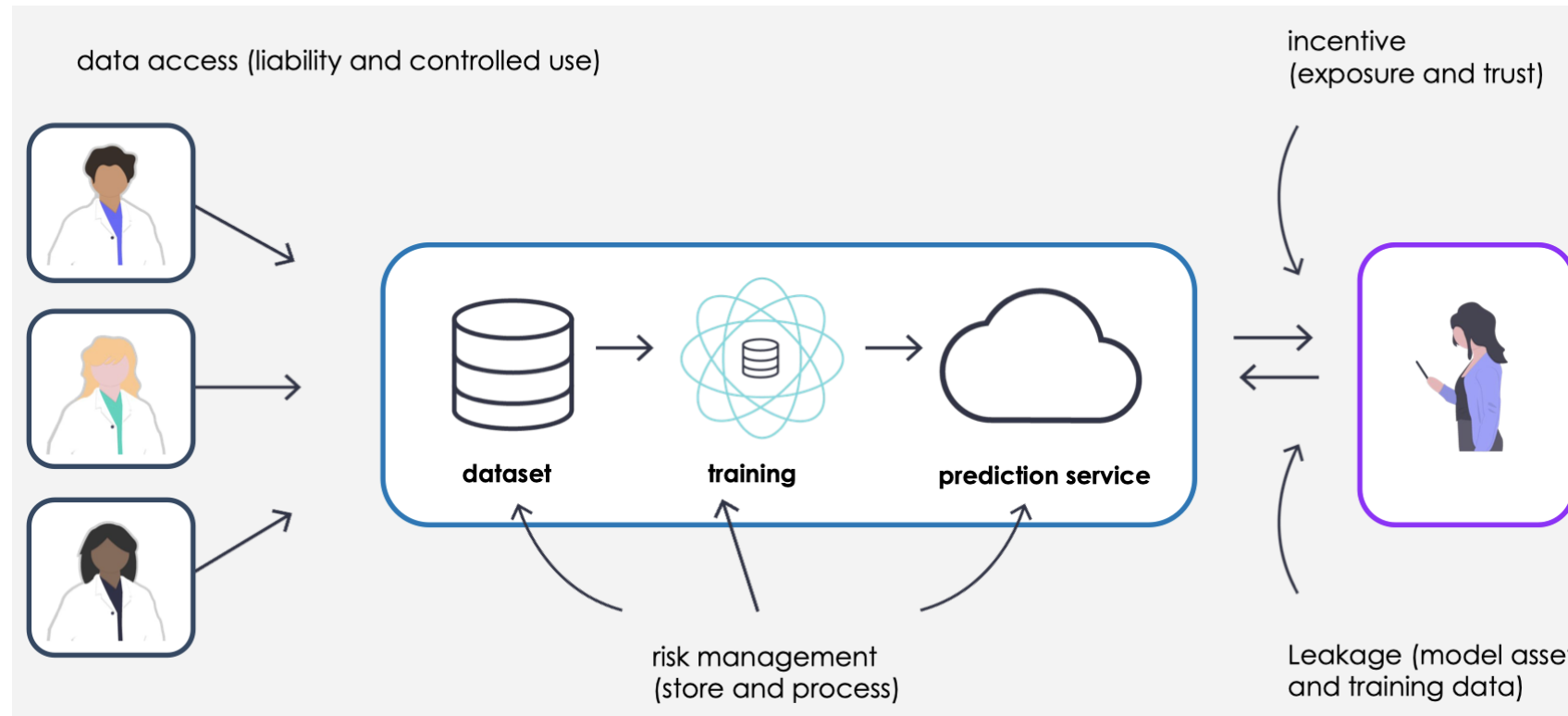


ML Workflow & Rủi ro bảo mật

Bottlenecks - Các nút thắt cổ chai trong học máy

Người dùng **không muốn chia sẻ** ảnh thật hoặc dữ liệu thật nếu không tin rằng hệ thống an toàn

→ Thiếu dữ liệu thực tế để huấn luyện.



Giải pháp: Privacy-Preserving Machine Learning với FHE

Agenda

- ✓ Giới thiệu
- **Cơ sở lý thuyết**
 - **PPML là gì?**
- Tổng quan nghiên cứu
- Phương pháp thực hiện
- Thực nghiệm
- Kết luận & hướng phát triển
- Nguồn tham khảo



Privacy-Preserving ML (PPML)

Telling stories: Alice and Bob



Alice



Bob

- *Alice:* Take this suitcase and please count the money
- *Bob:* Sure! But it is locked. Please, give me key.
- *Alice:* I am sorry. I can't trust you. You have to count it without opening it.
- *Bob:* Then, it can't be done! ...unless we use Privacy-Preserving Computation!

Privacy-Preserving ML (PPML)

Privacy-Preserving ML (PPML) là gì?

- **Tình huống:**
 - Một số người dùng/tổ chức có 1 số dữ liệu nhạy cảm.
 - Cần thực hiện một hoặc nhiều phép tính (ví dụ: thống kê, dự đoán, huấn luyện mô hình) mà không tiết lộ dữ liệu gốc cho bên thực hiện tính toán.

Mục tiêu:

- Dự đoán mà không tiết lộ dữ liệu người dùng.
- Huấn luyện mà không cần tập trung dữ liệu.

Tình huống thực tế:

- Chẩn đoán y tế từ ảnh không lộ ảnh.
- Dự đoán tài chính mà không lộ dữ liệu giao dịch.

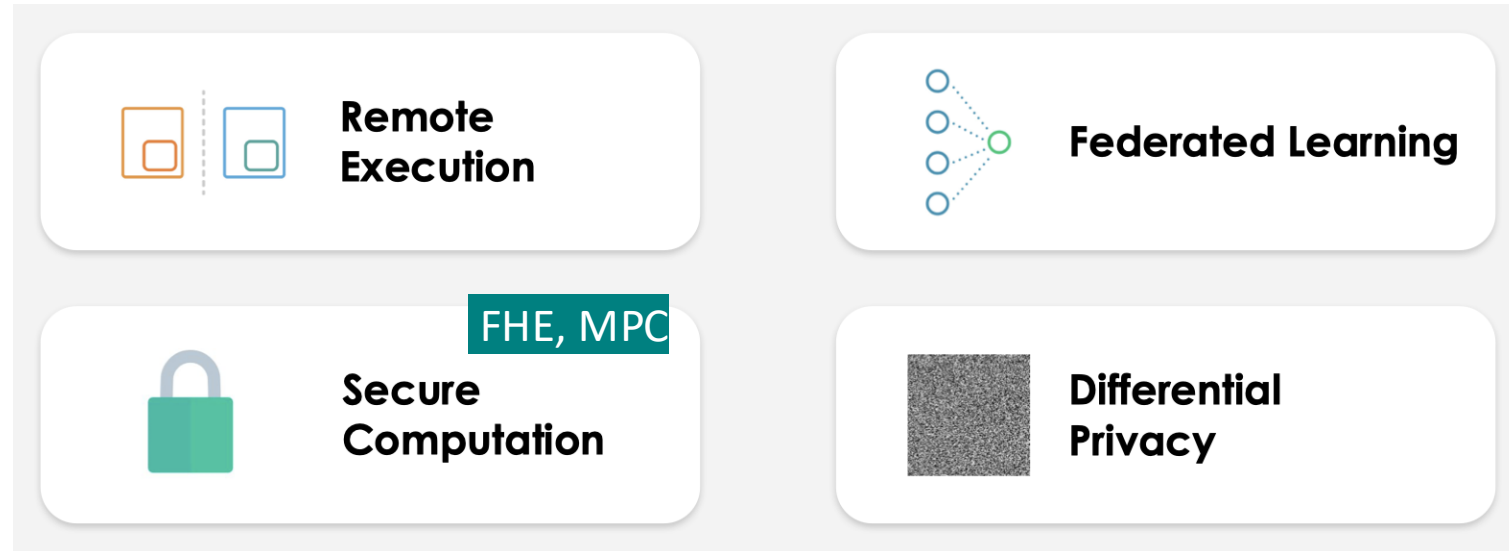


- **Alice:** Take these encrypted files. Please sum all the numbers in them.
- **Bob:** Sure! Please, give me decryption key.
- **Alice:** I am sorry. I can't trust you. You have to sum them without decrypting the files.
- **Bob:** Hummmmm... let me think...

Privacy-Preserving ML (PPML)

Các kỹ thuật Privacy-Preserving ML

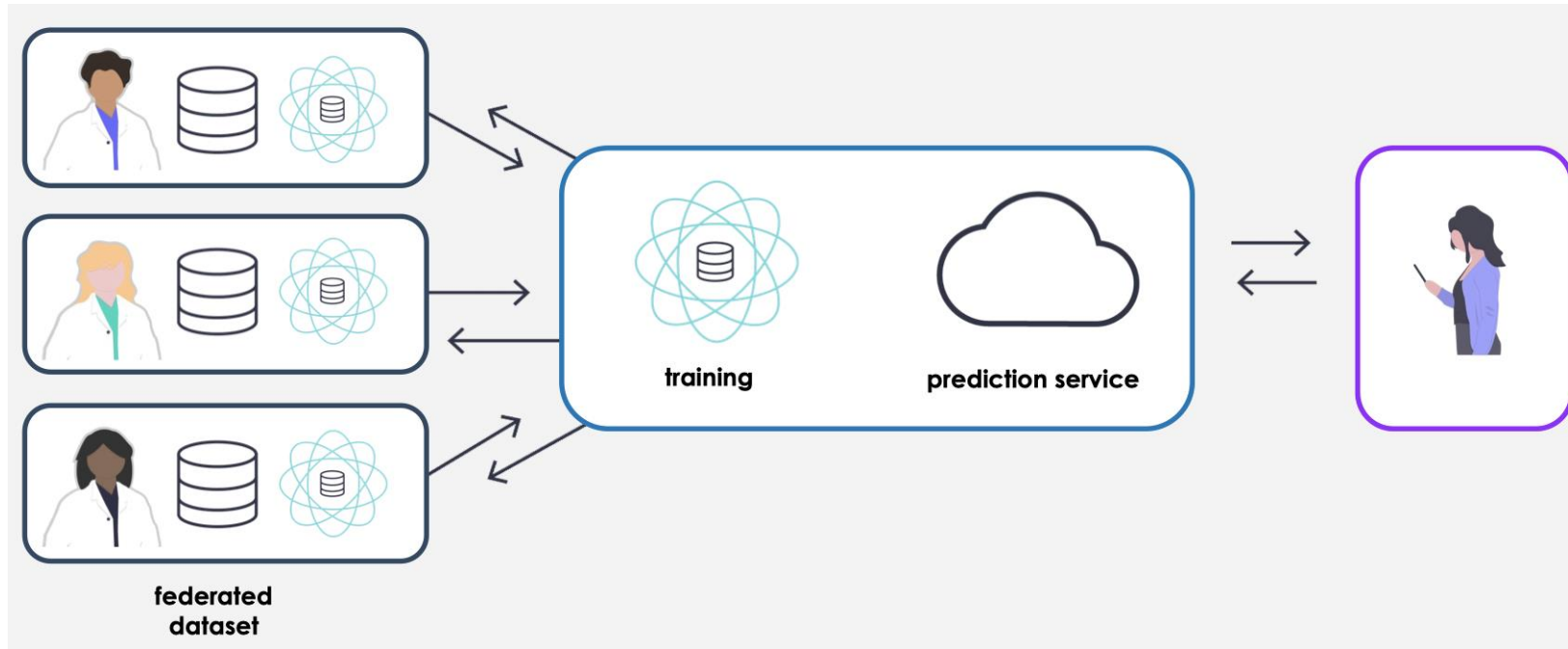
- Remote Execution – Chạy mô hình trên máy chủ từ xa mà không chia sẻ dữ liệu gốc.
- Federated Learning – Huấn luyện mô hình phân tán mà dữ liệu ở lại tại chỗ.
- Secure Computation – Tính toán bảo mật, có thể dùng FHE, MPC.
- Differential Privacy – Thêm nhiễu để bảo vệ dữ liệu cá nhân.



Privacy-Preserving ML (PPML)

Federated Learning

Federated Learning: Cho phép mô hình học **trực tiếp tại nơi dữ liệu được tạo ra** (ví dụ: bệnh viện, điện thoại người dùng), thay vì gửi dữ liệu lên server trung tâm.

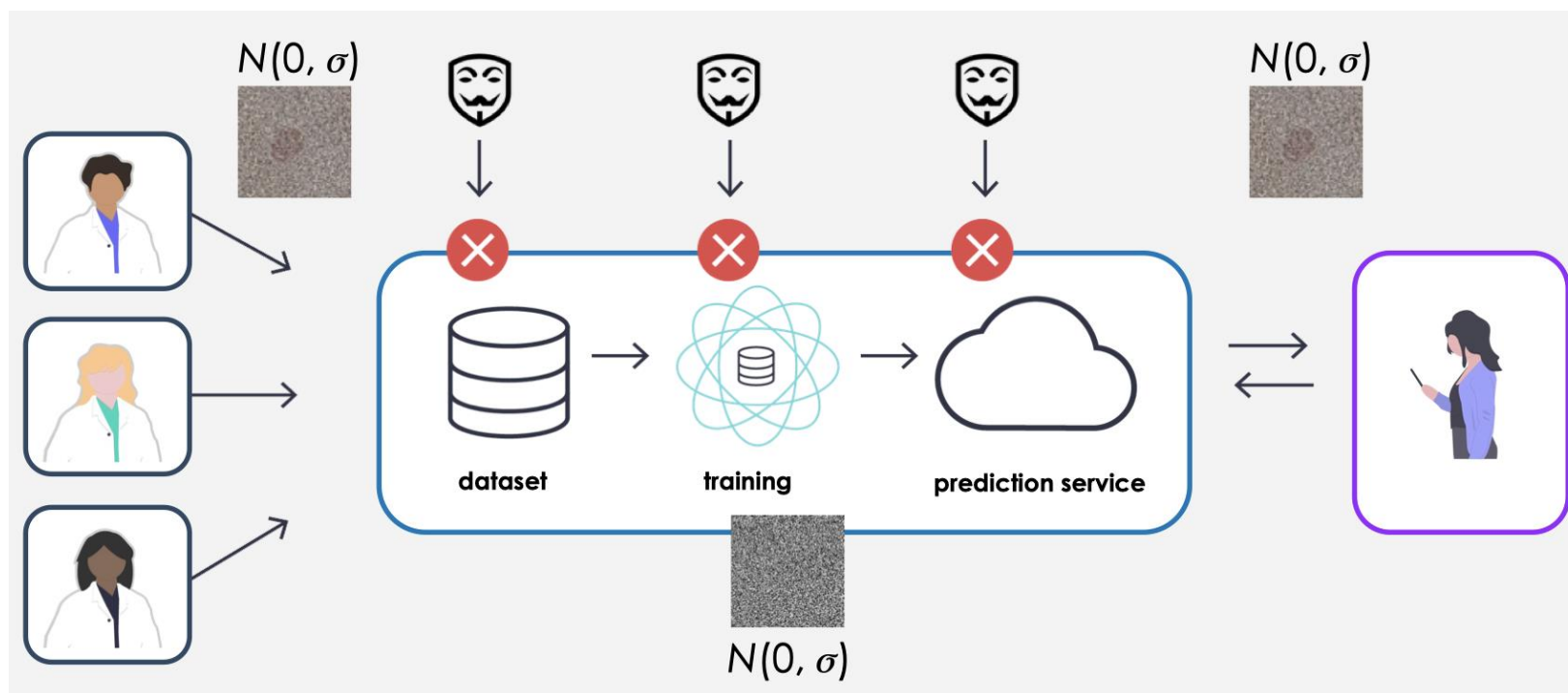


Privacy-Preserving ML (PPML)

Differential Privacy

Differential Privacy (DP): Bảo vệ quyền riêng tư trong học máy bằng cách thêm **nhiều ngẫu nhiên** (noise) vào dữ liệu hoặc quá trình huấn luyện.

Thêm nhiễu → tăng bảo mật nhưng **giảm độ chính xác**.



Privacy-Preserving ML (PPML)

So sánh các kỹ thuật Privacy-Preserving ML

Công nghệ	Dữ liệu rời nơi lưu trữ?	Có mã hóa không?	Mục tiêu chính	Ưu điểm / Hạn chế
Remote Execution	✗ Không	Có thể có	Chạy mô hình tại chỗ	Dễ triển khai, nhưng giới hạn môi trường chạy
Federated Learning	✗ Không	Thường không	Huấn luyện mô hình phân tán	Giữ dữ liệu tại chỗ, nhưng rò rỉ từ gradient
Secure Computation (FHE, MPC)	✓ Có (mã hóa)	✓ Có	Tính toán mà không thấy dữ liệu	Bảo mật cao, nhưng hiệu năng thấp
Differential Privacy	✓ Có	Không bắt buộc	Ngăn mô hình ghi nhớ cá nhân	Hiệu năng tốt, nhưng giảm độ chính xác

Fully Homomorphic Encryption (FHE) thuộc nhóm **Secure Computation**, cho phép tính toán trên dữ liệu mã hóa với độ bảo mật cao nhất.

Agenda

- ✓ Giới thiệu
- **Cơ sở lý thuyết**
 - ✓ PPML là gì?
 - **FHE nền tảng**
- Tổng quan nghiên cứu
- Phương pháp thực hiện
- Thực nghiệm
- Kết luận & hướng phát triển
- Nguồn tham khảo



Fully Homomorphic Encryption (FHE)

Nền tảng: Symmetric key cryptography

Khoá đối xứng:

- Chỉ dùng **1 khoá bí mật duy nhất** để mã hoá và giải mã.
- Đây là phương pháp **duy nhất được biết** đến cho đến năm 1976.

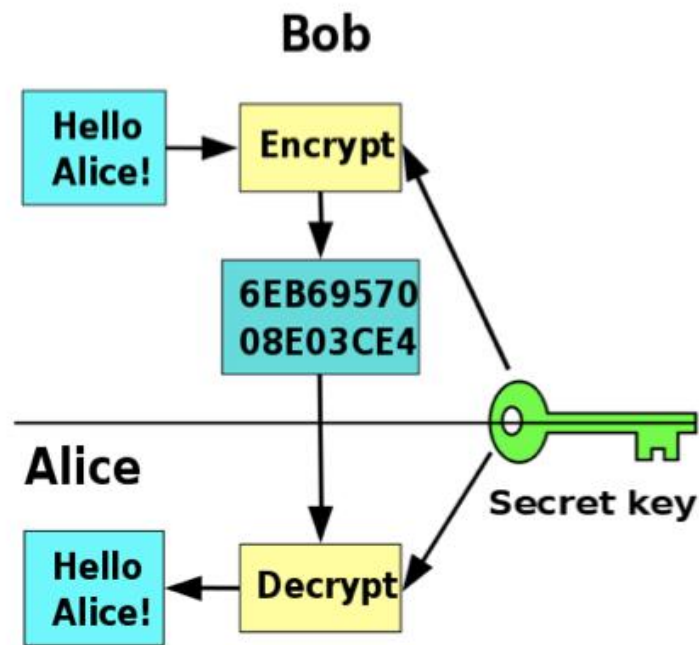
Vấn đề quản lý khoá:

- **Gửi khoá cho người nhận** một cách an toàn là một thách thức.
- Nếu mạng có nhiều người, số lượng khoá cần thiết tăng rất nhanh.

Bước ngoặt lịch sử (1976):

- Whitfield Diffie và Martin Hellman đề xuất **mật mã khoá công khai** (Asymmetric cryptography), mở ra cách giải quyết vấn đề phân phối khoá.

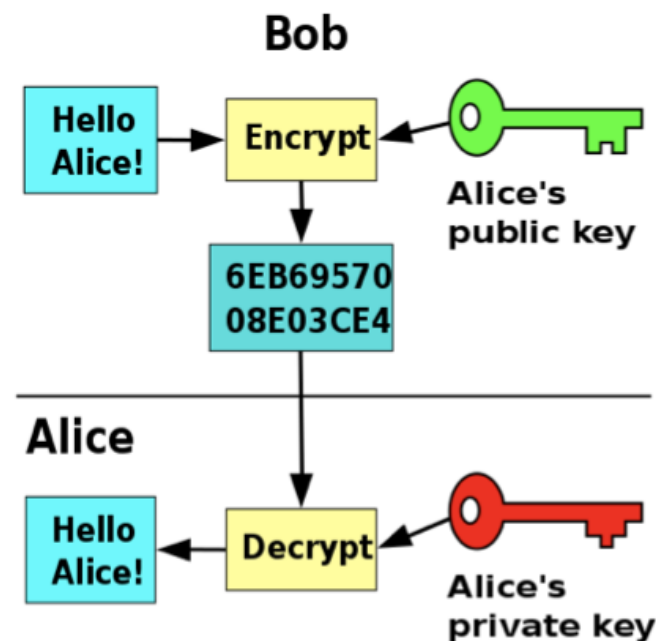
Khó khăn khi phân phối khoá



Fully Homomorphic Encryption (FHE)

Asymmetric key cryptography (tiền đề của FHE)

- **Khóa mã hóa** (Encryption key) là **khóa công khai** (public key) → mọi người đều có thể biết.
- **Khóa giải mã** (Decryption key) là **khóa bí mật** (private key) → chỉ chủ sở hữu giữ.
- Hai khóa liên quan về mặt toán học, nhưng không khả thi để suy ra khóa bí mật từ khóa công khai.
- Public key có thể phát tán tự do mà không làm lộ khóa bí mật, không lo bị lộ thông tin đủ để giải mã.
- Diffie–Hellman (1976) đưa ra giao thức chia sẻ khóa (key exchange protocol), nhưng chưa hỗ trợ mã hoá/giải mã dữ liệu.
- RSA (1978) của Rivest, Shamir, Adleman là hệ thống hoạt động đầy đủ đầu tiên dựa trên khóa công khai.
- Ứng dụng phổ biến: SSL/TLS, VPN, chữ ký số.



Giải quyết **phân phối khóa**, đặt nền tảng cho các hệ mật mã có tính chất đồng hình (Homomorphic)

Fully Homomorphic Encryption (FHE)

Homomorphisms – nền tảng của FHE

Để hiểu Homomorphic Encryption, trước hết cần khái niệm toán học về Homomorphism – khả năng bảo toàn phép toán sau khi biến đổi.

Định nghĩa đại số

- Homomorphism: là một ánh xạ bảo toàn cấu trúc giữa hai cấu trúc đại số cùng loại (hai nhóm, hai vành, v.v.).
- Nói cách khác: nếu bạn có phép toán trong cấu trúc ban đầu, phép toán tương ứng cũng đúng trong cấu trúc đích sau khi biến đổi.

Ví dụ minh họa

- G1: Tập số thực với phép cộng (+).
 - G2: Tập số thực dương với phép nhân (×).
 - Hàm $f(x)=e^x$ là một homomorphism từ G1 sang G2 vì:
$$f(x+y)=e^{x+y}=e^x \cdot e^y$$
- phép cộng trong G1 tương đương với phép nhân trong G2.

Homomorphisms Encryption

Trong Homomorphisms Encryption, hàm mã hóa Enc đóng vai trò như một homomorphism:

- Nếu lược đồ là cộng đồng hình:
$$\text{Enc}(a+b)=\text{Enc}(a) \oplus \text{Enc}(b)$$
- Nếu lược đồ là nhân đồng hình:
$$\text{Enc}(a \cdot b)=\text{Enc}(a) \otimes \text{Enc}(b)$$
- Trong FHE, cả hai tính chất này đều được bảo toàn → ta có thể thực hiện cả cộng và nhân trên dữ liệu đã mã hóa.

Đây chính là lý do FHE “tính toán trên ciphertext” mà vẫn cho kết quả đúng sau khi giải mã.

Homomorphism: phép toán trên **dữ liệu gốc** được phản ánh tương ứng trên **dữ liệu đã biến đổi** (ciphertext).

Fully Homomorphic Encryption (FHE)

Ví dụ siêu đơn giản (chưa đầy đủ bảo mật, chỉ để hiểu ý tưởng)

Giả sử ta mã hóa một số m như sau:

$$Enc(m) = m + k \cdot N$$

trong đó:

- k là số ngẫu nhiên (tạo nhiễu),
- N là một số lớn (modulus),
- chỉ người giải mã mới biết N .

Phép cộng trên bản mã:

$$\begin{aligned} Enc(m_1) + Enc(m_2) &= (m_1 + k_1N) + (m_2 + k_2N) \\ &= (m_1 + m_2) + (k_1 + k_2)N \end{aligned}$$

→ Khi giải mã (lấy mod N), ta nhận lại $m_1 + m_2$.

1. Thiết lập tham số

- Chọn **modulus** $N = 97$ (người giải mã biết N).
- Chọn ngẫu nhiên số k để sinh nhiễu.

2. Mã hóa hai số

Giả sử: $m_1 = 15$, $m_2 = 27$

Chọn:

$$k_1 = 3 \rightarrow Enc(m_1) = 15 + 3 \cdot 97 = 15 + 291 = 306$$

$$k_2 = 5 \rightarrow Enc(m_2) = 27 + 5 \cdot 97 = 27 + 485 = 512$$

Ta có: $c_1 = 306$, $c_2 = 512$

3. Cộng trên bản mã

$$c_{sum} = c_1 + c_2 = 306 + 512 = 818$$

4. Giải mã kết quả

$$Dec(c_{sum}) = 818 \bmod 97$$

$$97 \times 8 = 776$$

$$818 - 776 = 42$$

→ Kết quả giải mã: **42**.

5. So sánh với phép cộng bản rõ

$$m_1 + m_2 = 15 + 27 = 42$$

Fully Homomorphic Encryption (FHE)

Partially Homomorphic Encryption (PHE) – Đồng hình một phần

- Chỉ hỗ trợ không giới hạn **một loại phép toán duy nhất** (cộng hoặc nhân).
- Ví dụ:
 - Unpadded RSA, ElGamal: đồng hình nhân.
 - Goldwasser-Micali, Benaloh, Paillier: đồng hình cộng.
- Ứng dụng: Thống kê bảo mật (cộng dữ liệu mã hóa), secure aggregation trong FL.

Somewhat Homomorphic Encryption (SHE) – Đồng hình hạn chế

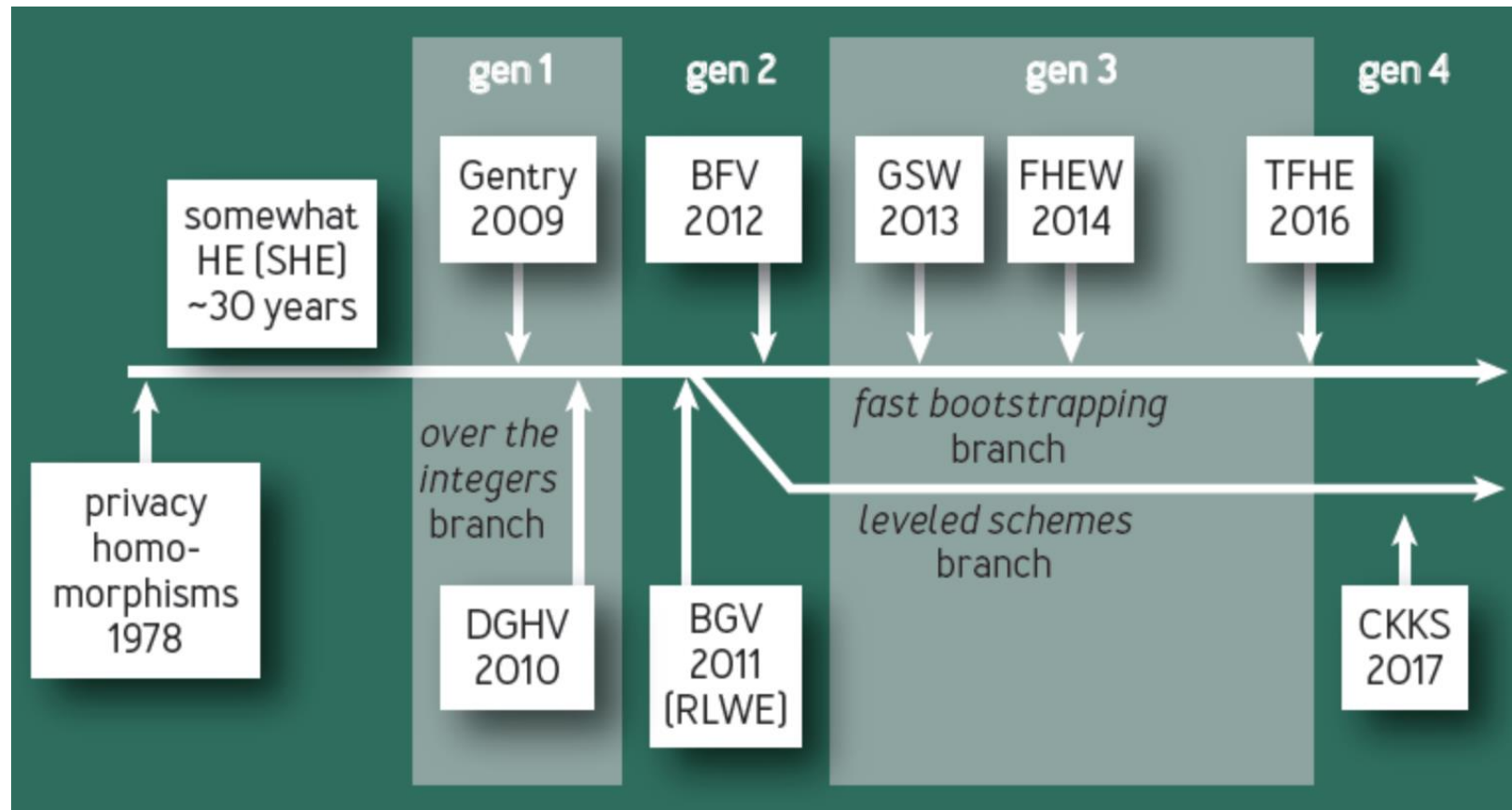
- Hỗ trợ cả cộng và nhân, nhưng **bị giới hạn số lần phép toán** do nhiễu trong ciphertext.
- Ví dụ:
 - Boneh-Goh-Nissim (BGN), LTV (López-Alt, Tromer, Vaikuntanathan), BFV
- Ứng dụng: Các bài toán tính toán bảo mật nhỏ, dùng cho PoC nhưng không đủ mạnh cho ứng dụng thực tế vì noise tăng nhanh.

Fully Homomorphic Encryption (FHE) – Đồng hình hoàn toàn

- Hỗ trợ cộng và nhân không giới hạn → tính toán được bất kỳ hàm nào trên dữ liệu mã hóa.
- Ví dụ:
 - Lược đồ FHE của Craig Gentry (2009) – nền tảng lý thuyết đầu tiên.
 - Các lược đồ thế hệ 2: BGV, BFV, CKKS, TFHE...
- Ứng dụng: Inference an toàn trên cloud, xử lý dữ liệu y tế, tài chính trong môi trường zero-trust.

Fully Homomorphic Encryption (FHE)

FHE Generations



Fully Homomorphic Encryption (FHE)

Giải thích FHE (public-key case)

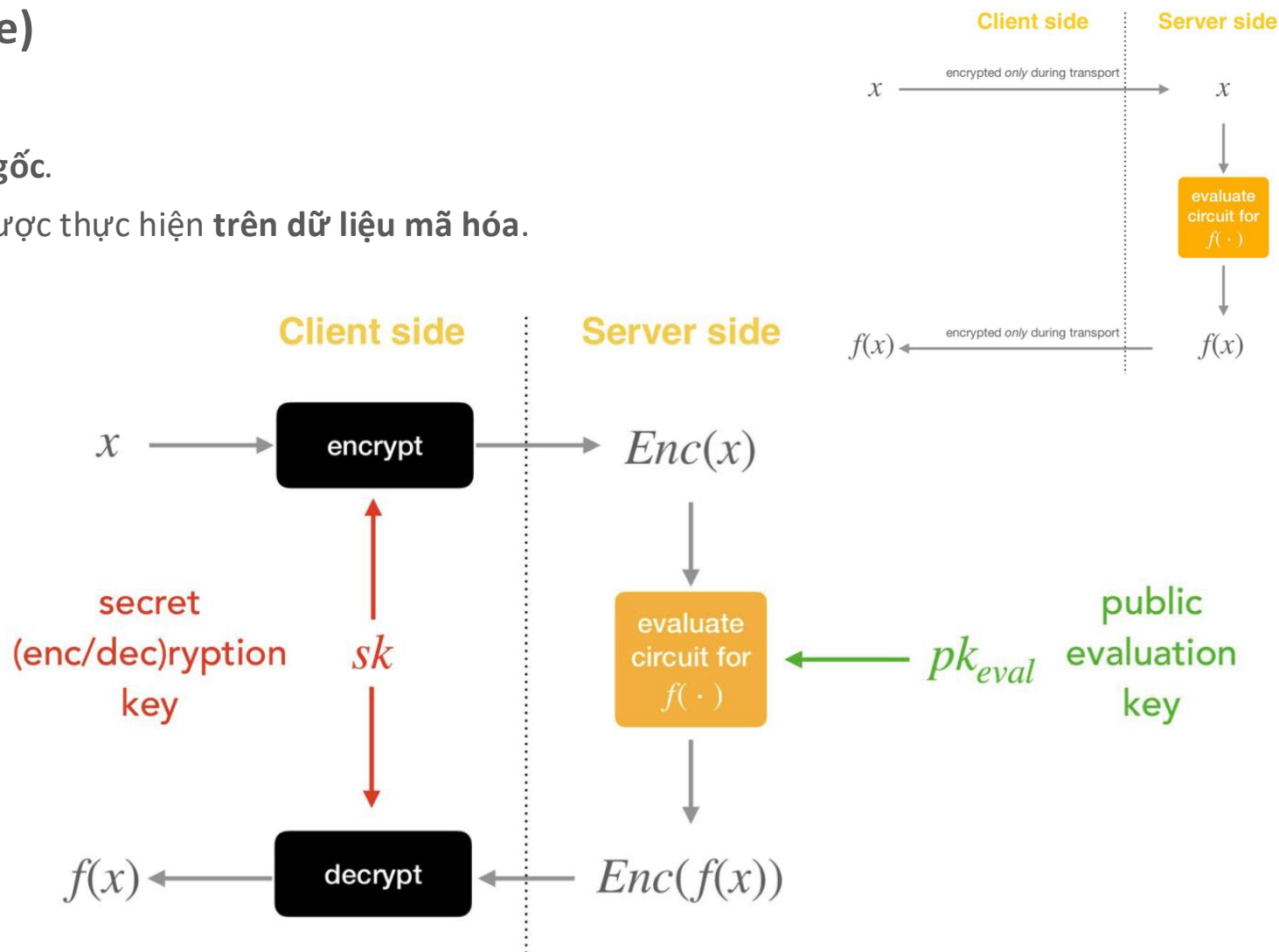
Điểm quan trọng:

- Server không bao giờ thấy dữ liệu gốc.
- Việc tính toán (cộng, nhân,...) vẫn được thực hiện **trên dữ liệu mã hóa**.

Vai trò của khoá:

- sk: client giữ, chỉ để giải mã
- pk: client dùng để mã hoá
- Eval keys: server dùng để Eval, không giải mã được.

$$\text{Dec}_{sk}(\text{Eval}_{evk}(f, \text{Enc}_{pk}(x))) = f(x)$$



Fully Homomorphic Encryption (FHE)

Ví dụ cách hoạt động của FHE

Giả sử:

- **Client** muốn tính $(x+y)*z$ với $x=5, y=3, z=2$, nhưng không muốn lộ x,y,z .

Quy trình:

1. Client sinh khoá, mã hoá:

$$E_x = \text{Enc}(5), E_y = \text{Enc}(3), E_z = \text{Enc}(2)$$

2. Gửi E_x, E_y, E_z cho server.

3. Server tính:

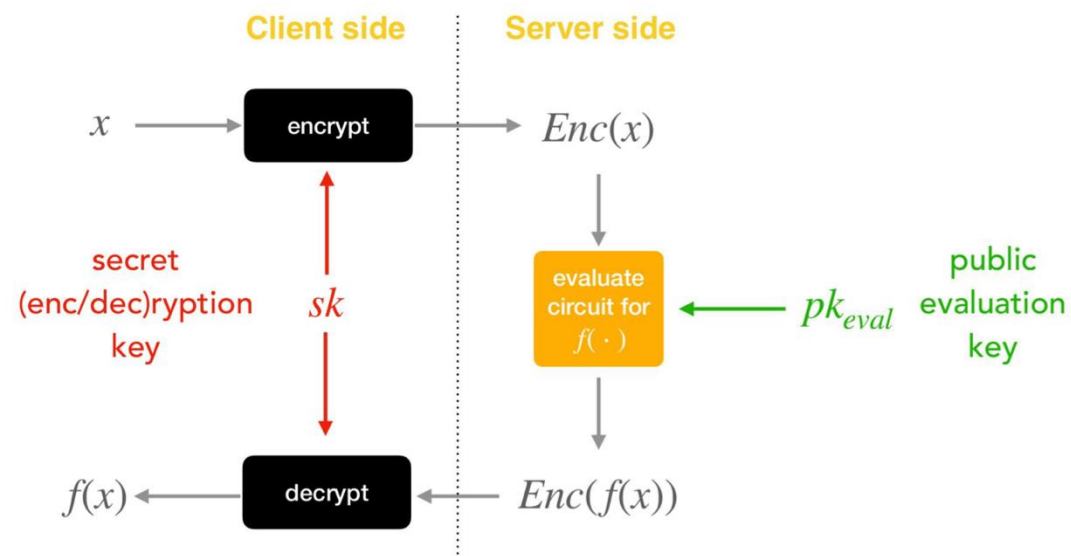
$$E_{\text{sum}} = E_x + E_y \text{ (FHE cộng)}$$

$$E_{\text{result}} = E_{\text{sum}} * E_z \text{ (FHE nhân)}$$

4. Trả E_{result} cho client.

5. Client giải mã:

$$\text{Dec}(E_{\text{result}}) = (5+3)*2 = 16$$



Giá trị gốc	Sau mã hóa	Phép tính trên ciphertext	Kết quả giải mã
(5,3,2)	(E_x, E_y, E_z)	$(E_x + E_y) * E_z$	16

Fully Homomorphic Encryption (FHE)

So sánh FHE và E2EE (End-to-End Encryption)

Quy trình xử lý dữ liệu

Bước	E2EE	FHE
1. Mã hóa	Người gửi mã hóa dữ liệu trên thiết bị của mình.	Người gửi mã hóa dữ liệu trên thiết bị của mình.
2. Truyền / Lưu trữ	Dữ liệu di chuyển qua mạng hoặc lưu trữ ở server dưới dạng mã hóa, không ai đọc được ngoài người nhận.	Dữ liệu được đưa đến server hoặc dịch vụ xử lý ở dạng mã hóa.
3. Xử lý dữ liệu	Không xử lý được khi còn mã hóa — nếu muốn xử lý, phải giải mã (→ mất an toàn).	Có thể xử lý trực tiếp ngay trên dữ liệu đã mã hóa (tính toán, phân tích, AI...).
4. Giải mã	Người nhận giải mã để đọc dữ liệu.	Sau khi xử lý xong, người nhận giải mã để nhận kết quả đã tính toán mà không bao giờ lộ dữ liệu gốc cho bên trung gian.

Bảo vệ tính riêng tư khi truyền tải dữ liệu. Đảm bảo chỉ người gửi và người nhận đọc được dữ liệu. Không nhằm mục đích xử lý dữ liệu ở trạng thái mã hóa.	Bảo vệ tính riêng tư trong khi xử lý dữ liệu. Cho phép máy chủ hoặc dịch vụ bên thứ ba thực hiện tính toán trên dữ liệu mà không cần giải mã.
E2EE	FHE

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- **Tổng quan nghiên cứu**
- Phương pháp thực hiện
- Thực nghiệm
- Kết luận & hướng phát triển
- Nguồn tham khảo



Tổng quan nghiên cứu

Privacy Preserving Machine Learning with Homomorphic Encryption and Federated Learning

Fang, Haokun, and Quan Qian. Future Internet (2021)

HE dùng trong training (FL + HE cho aggregation)

Mục tiêu nghiên cứu

Đề xuất một framework PFMLP kết hợp Homomorphic Encryption (HE) và Federated Learning (FL) để:

- Cho phép huấn luyện mô hình ML phân tán mà không cần chia sẻ dữ liệu gốc. Dùng Paillier Homomorphic Encryption.
- Đảm bảo tính riêng tư trong quá trình truyền và tổng hợp tham số mô hình (weights/gradients).

Giải quyết vấn đề: mặc dù FL đã giúp dữ liệu không rời khỏi thiết bị, nhưng gradients gửi về server vẫn có thể bị tấn công → cần thêm HE để mã hóa gradients.

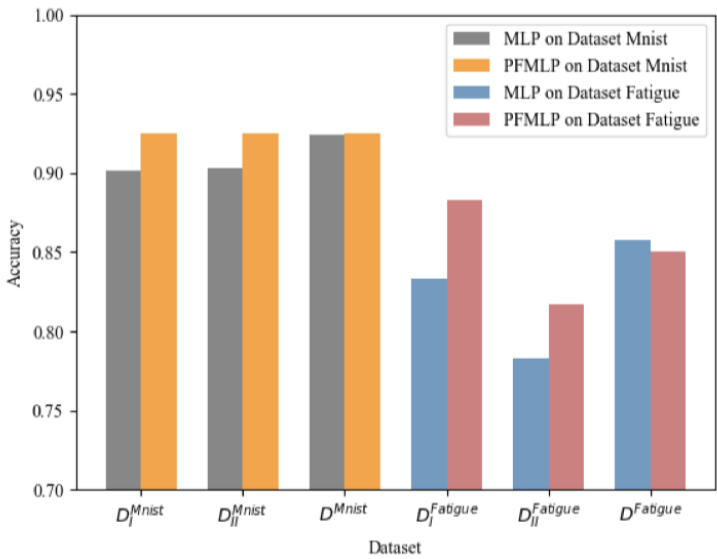


Figure. Prediction accuracy between MLP and PFMLP

Encryption Algorithm	Dataset	Key Length (Bits)	Time (Seconds)
None	MNIST	-	7.92
Paillier		128	12,033.25 (~3.3h)
Paillier		256	45,467.44 (~12.6h)
Paillier		512	199,915.70 (~55.5h)
None	Fatigue	-	9.84
Paillier		64	2567.44 (~42m)
Paillier		128	4480.02 (~1.2h)
Paillier		256	13,428.92 (~3.7h)

Table. Impact of different key lengths on total training time

Tổng quan nghiên cứu

Privacy-Preserving Machine Learning with Fully Homomorphic Encryption for Deep Neural Network

Lee, Joon-Woo, et al. iEEE Access (2022)

Inference bằng FHE

Mục tiêu nghiên cứu

- Đề xuất một framework sử dụng Fully Homomorphic Encryption (FHE) để triển khai deep neural networks (DNNs) trên dữ liệu đã mã hoá. Các tác giả triển khai mô hình ResNet-20 chuẩn với RNS-CKKS FHE kèm bootstrapping, SEAL library.
- Giải quyết bài toán: làm sao để có thể thực hiện inference ML trên dữ liệu nhạy cảm mà vẫn giữ độ chính xác cao và hiệu năng chấp nhận được.
- Nâng cao mức riêng tư so với các kỹ thuật khác như PHE, SHE hoặc chỉ FL + HE.

Model	ResNet-20 ¹	ResNet-20 ²	PPML ResNet-20	Agreement
Accuracy	91.89%	92.95% $\pm$ 2.56%	92.43% $\pm$ 2.65%	98.43% $\pm$ 1.25%

Figure. Classification accuracy of the ResNet-20 for plaintext and ciphertext and agreement ratio

Layer	Conv	BN	ReLU	Boot	AP + FC	Softmax	Total time (s)
Time ratio	17.44%	13.55%	34.61%	31.55%	0.04%	2.81%	10,602 (~2.94h)

The running time of the ResNet-20 and the percentage of time spent in each component relative to total time

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- ✓ Tổng quan nghiên cứu
 - **Phương pháp thực hiện**
 - **Tích hợp FHE vào ML**
- Thực nghiệm
- Kết luận & hướng phát triển
- Nguồn tham khảo

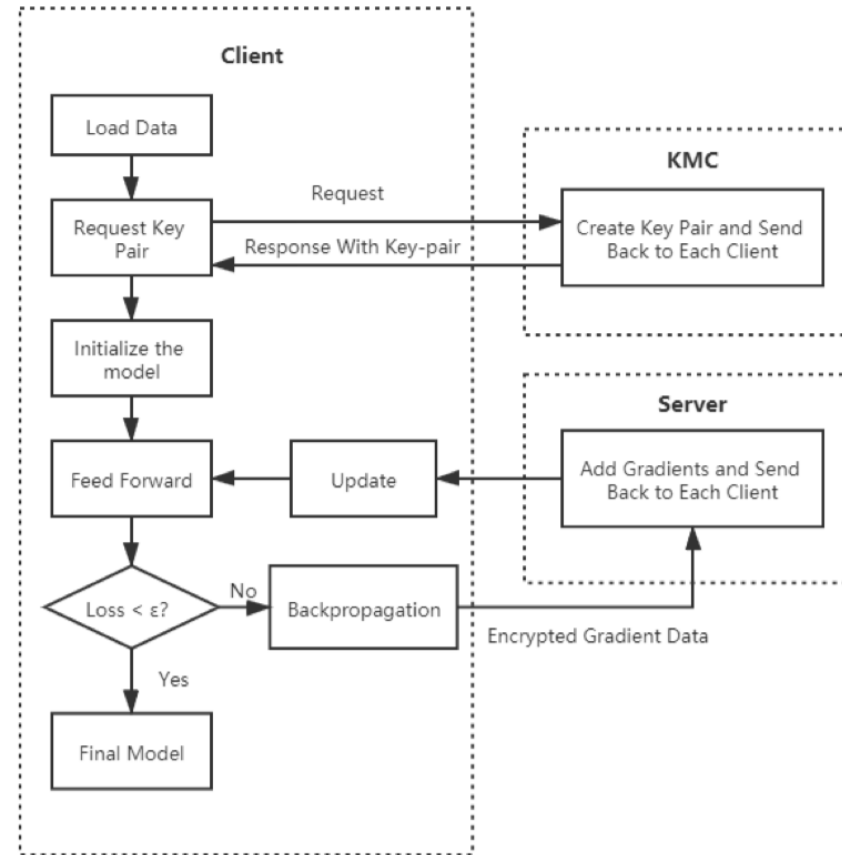
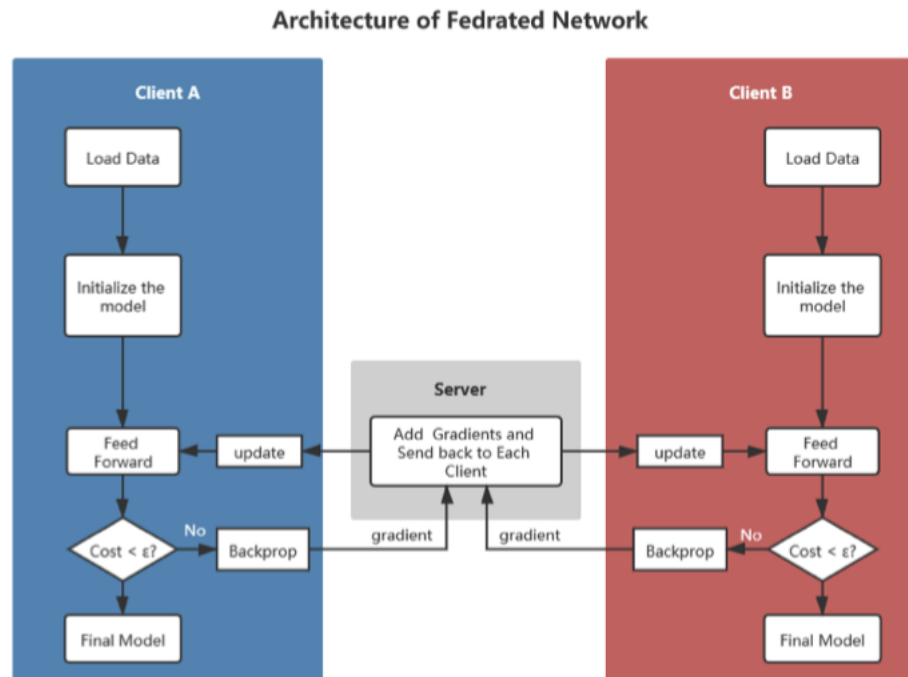


Phương pháp thực hiện

Tích hợp FHE vào training ML/DL

FHE thuần training → quá chậm, khó áp dụng thực tế.

Giải pháp: kết hợp với FL / MPC / HE khác để làm khả thi hơn.



The architecture of a Paillier federated network

Training bằng FHE thuần hiện tại còn hạn chế, chủ yếu inference được ứng dụng trong thực tế; training thường cần kết hợp HE + MPC/FL.

Phương pháp thực hiện

Tích hợp FHE vào Inference ML/DL

Các bước thực hiện:

1. Client gửi dữ liệu mã hoá → Server.
2. Server tính toán và trả về kết quả mã hóa.
3. Client giải mã kết quả dự đoán.



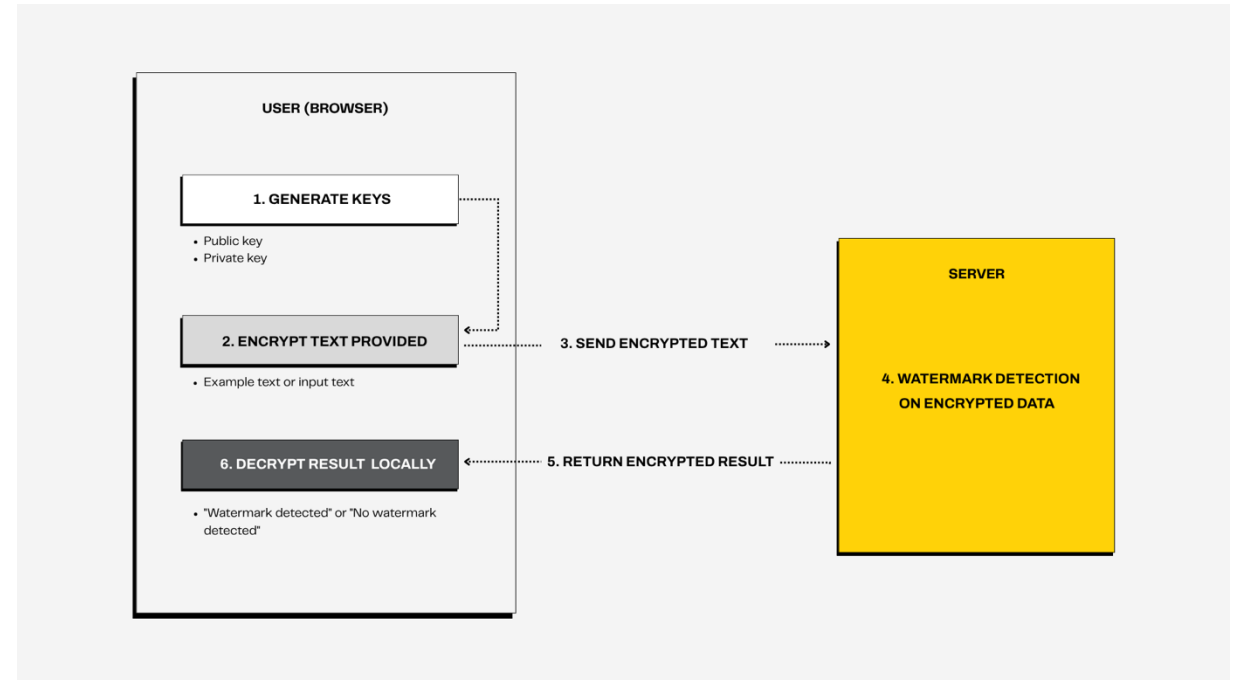
Inference xảy ra hoàn toàn trên dữ liệu mã hoá

Ví dụ:

Bác sĩ gửi ảnh X-ray đã mã hóa → Cloud AI → trả về kết quả chẩn đoán mã hóa → Giải mã tại máy bác sĩ.

Pipeline minh họa:

model (clear) → quantize/approx → compile to circuit → client encrypt → server evaluate (FHE) → client decrypt



Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- ✓ Tổng quan nghiên cứu
- **Phương pháp thực hiện**
 - ✓ Tích hợp FHE vào ML
 - **Concrete-ML**
- Thực nghiệm
- Kết luận & hướng phát triển
- Nguồn tham khảo



Giới thiệu Concrete-ML

Concrete-ML là thư viện mã nguồn mở do Zama phát triển, cho phép **huấn luyện (cleartext)** và **suy luận trên dữ liệu mã hóa hoàn toàn (FHE)** mà không cần kiến thức mật mã học chuyên sâu.

Hỗ trợ tích hợp trực tiếp với scikit-learn, PyTorch, TensorFlow, ONNX.



with classical toolbox: **fit**, **predict**, **pipeline**,
gridsearch

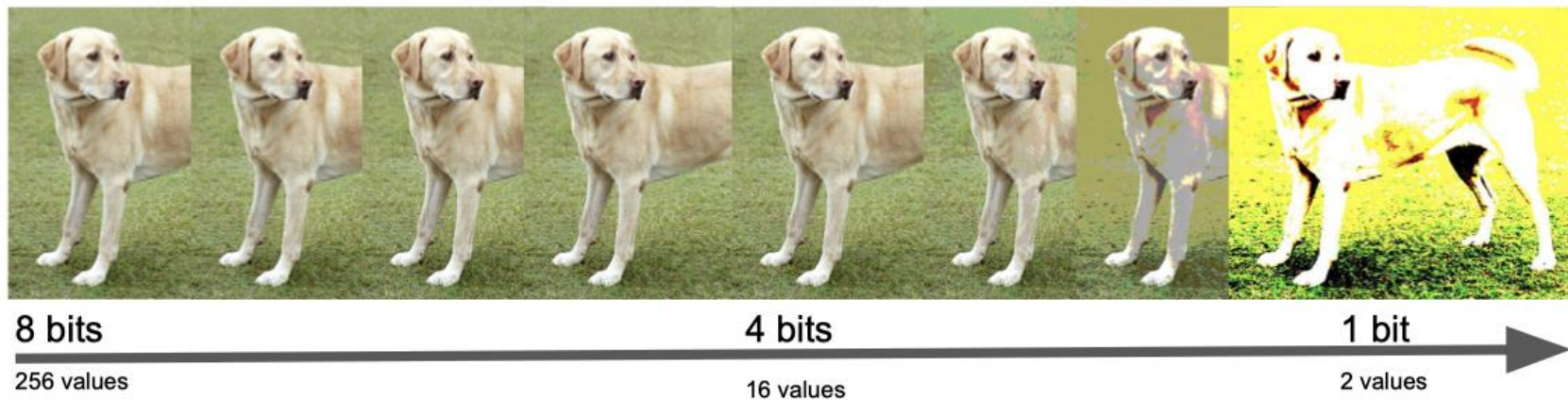
Computation with Integers only

Trong FHE, các phép toán **chỉ áp dụng được trên số nguyên** (integers).

Dữ liệu đầu vào (ví dụ: trọng số, đầu vào) là số thực, văn bản, hình ảnh thì cần được lượng tử hóa (quantization) để FHE xử lý.

Hình ảnh minh họa:

- Khi lượng tử hóa từ 8 bits (256 giá trị) xuống 4 bits (16 giá trị) rồi 1 bit (2 giá trị), chất lượng ảnh giảm dần.
- Giảm số bit sẽ giảm độ chính xác, nhưng giúp giảm độ phức tạp tính toán trong FHE.



Các mô hình tích hợp sẵn trong Concrete-ML

Concrete-ML cung cấp nhiều thuật toán machine learning đã được tối ưu để chạy trên dữ liệu mã hóa bằng FHE.

Tree Based Models

Decision Trees
Random Forest
XGBoost

Linear Models

Linear Regression
Logistic Regression
Generalized Linear Models
SVM
ElasticNet
Lasso
Ridge

Neural Networks

Multi-Layer Perceptron
Custom Neural Networks

- Torch,
- Tensorflow,
- ONNX

Phương pháp thực hiện

Pipeline triển khai Inference Concrete-ML

Quy trình triển khai Concrete-ML:

Data

↓ (Quantization)

Integers

↓ (Encrypt)

Encrypted Data

↓ (Inference in FHE)

Encrypted Result

↓ (Decrypt)

Output

Ưu điểm

- Dễ sử dụng, không cần kiến thức mật mã
- Tích hợp sẵn với nhiều framework ML
- Hỗ trợ nhiều mô hình (Tree, Linear, NN)
- Bảo vệ dữ liệu nhạy cảm

Giới hạn

- Tốc độ chậm hơn ML thông thường
- Giới hạn kích thước mô hình
- Yêu cầu lượng tử hóa dữ liệu

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- ✓ Tổng quan nghiên cứu
- ✓ Phương pháp thực hiện
 - **Thực nghiệm**
 - Kết luận & hướng phát triển
 - Nguồn tham khảo



Demo Concrete-ML cho PPML

Demo 1: MLP (Iris Classification)

Mục tiêu: Huấn luyện mô hình MLP để phân loại 3 loài hoa, sau đó biên dịch mô hình sang dạng có thể chạy trên dữ liệu đã mã hóa (FHE).

Quy trình triển khai

1. Xử lý dữ liệu

- Dữ liệu: `load_iris()` từ `sklearn`, gồm 3 lớp hoa.
- Thực hiện: Chia train/test (75%/25%). Chuyển dữ liệu sang `float32` (bắt buộc cho Concrete-ML).

2. Kiến trúc mạng Neural

- Sử dụng `NeuralNetClassifier` từ Concrete-ML.
- Mạng gồm: Input: 4 đặc trưng. Hidden layers: 2 lớp 16 neuron. Output: 3 lớp softmax (Iris-setosa, versicolor, virginica).
- Activation: ReLU (dùng quantized/LUT hay xấp xỉ đa thức).

3. Huấn luyện và đánh giá

- Huấn luyện mô hình trên dữ liệu huấn luyện.
- Đánh giá độ chính xác trên tập test.
- In ra kết quả `accuracy_score` ($\approx 96.6\%$).

4. Biên dịch sang FHE

- Concrete giúp chuyển đổi mô hình sang dạng có thể chạy trên dữ liệu mã hoá.

5. Inference trên dữ liệu mã hoá

- Dữ liệu đầu vào được mã hoá bằng FHE.
- Mô hình thực hiện inference trực tiếp trên ciphertext.
- Sau khi inference hoàn tất, kết quả được giải mã để so sánh.

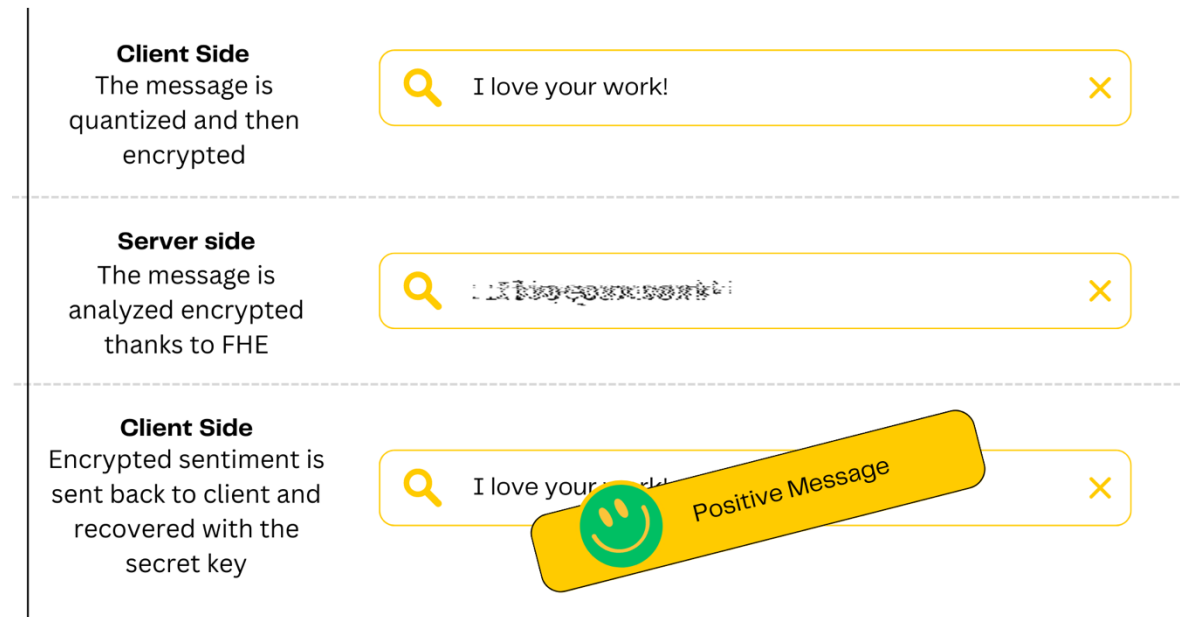
Demo Concrete-ML cho PPML

Demo 2: Sentiment Analysis

Cho phép phân tích cảm xúc (positive, neutral, negative) trên **dữ liệu mã hóa** mà **không cần giải mã** — bảo vệ riêng tư dữ liệu.

Công cụ & Thư viện chính

- **Concrete-ML**: thư viện cho phép chuyển đổi mô hình ML thành dạng có thể sử dụng trên dữ liệu mã hóa mà không cần kiến thức mật mã học.



Demo 2: Sentiment Analysis

Quy trình triển khai

1. **Cài đặt môi trường:** có thể chạy trên máy cá nhân hoặc trên Hugging Face.
2. **Chọn dữ liệu công khai:** bộ dữ liệu Twitter airline sentiment; tỷ lệ phân bố: positive ~16%, negative ~63%, neutral ~21%.
3. **Huấn luyện mô hình:**
 - Sử dụng XGBoost qua Concrete-ML XGBClassifier, tìm hyper-parameters với GridSearch.
 - Kết quả: accuracy ~84% (cross-validation), ~85% trên test set.
4. **Inference trên dữ liệu mã hoá (FHE):**
 - Compile mô hình để tạo engine FHE (~9.3s)
 - Inference FHE trên một tweet mất ~4.4s; kết quả giống với inference không mã hoá.
5. **Triển khai thực tế:**
 - Deployment: Tạo API (client, server) phục vụ FHE inference.
 - Demo: Tạo demo với chức năng:
 - Client: tạo khoá, mã hoá input.
 - Server: nhận dữ liệu mã hoá, tính toán, trả kết quả mã hoá.
 - Client cuối cùng nhận kết quả mã hoá, giải mã và hiển thị kết quả inference.

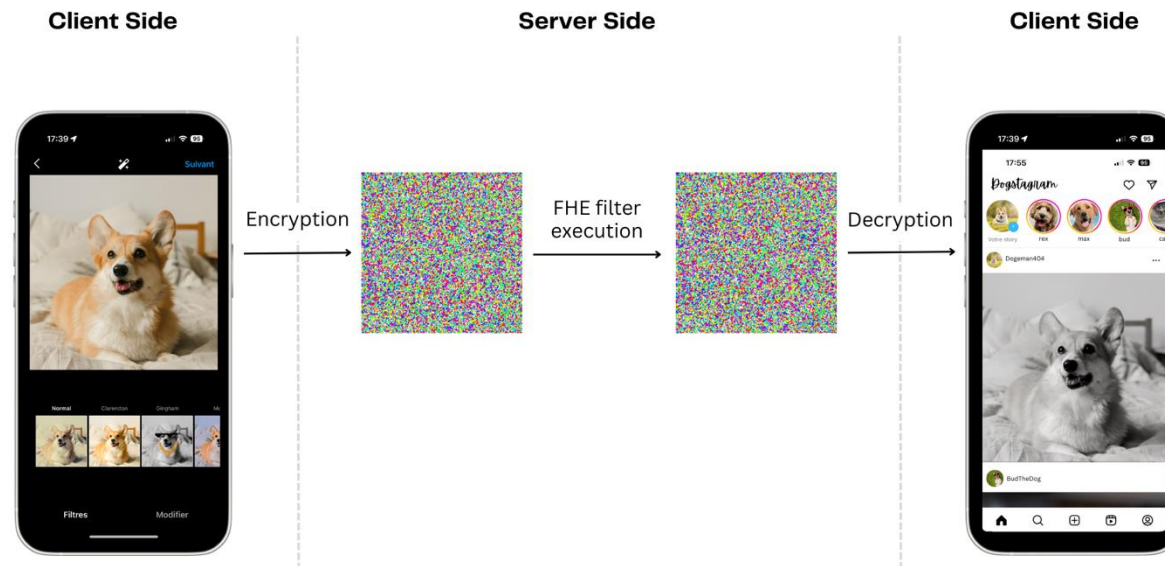
Demo dùng Concrete-ML

Demo 3: Image Filtering

Mục tiêu: Cho phép áp dụng các bộ lọc hình ảnh (như làm sắc nét, đảo màu, v.v.) trực tiếp trên dữ liệu đã mã hoá bằng Fully Homomorphic Encryption, mà không cần giải mã — đảm bảo bảo mật tuyệt đối cho dữ liệu hình ảnh

Công cụ & Thư viện chính

- **Concrete-ML:** thư viện cho phép chuyển đổi mô hình ML thành dạng có thể sử dụng trên dữ liệu mã hóa mà không cần kiến thức mật mã học.



Demo 3: Image Filtering

Pipeline Client – Server FHE

- Client:
 - Tạo khoá (genkey).
 - Mã hoá ảnh đầu vào.
 - Gửi ảnh mã hoá đến server.
- Server:
 - Nhận dữ liệu mã hoá.
 - Thực thi filter trên ciphertext bằng circuit FHE.
 - Trả về kết quả vẫn dưới dạng mã hoá.
- Client:
 - Giải mã (decrypt) để nhận ảnh đã xử lý – đảm bảo quyền riêng tư.

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- ✓ Tổng quan nghiên cứu
- ✓ Phương pháp thực hiện
- ✓ Thực nghiệm
- **Kết luận & hướng phát triển**
- Nguồn tham khảo



Kết luận & hướng phát triển

Kết luận

- FHE cung cấp khả năng **tính toán trực tiếp trên dữ liệu mã hóa** → bảo mật tối đa, ngay cả trong môi trường không tin cậy.
- Trong ML/DL, FHE phù hợp nhất cho **inference-as-a-service** trên dữ liệu nhạy cảm (y tế, tài chính, quốc phòng...).
- **Concrete-ML** giúp triển khai inference trên FHE mà không cần kiến thức sâu về mật mã.
- Kết quả paper và demo (Iris Classification, Sentiment Analysis, Image Filtering) → **độ chính xác tương đương** với inference trên dữ liệu rõ.
- **Hạn chế hiện tại**: tốc độ chậm, tốn bộ nhớ, khó áp dụng cho deep learning phức tạp hoặc training trực tiếp trên dữ liệu mã hóa.
- FHE là bước tiến quan trọng trong **bảo vệ quyền riêng tư cho học máy**, đặc biệt trong môi trường **zero-trust**.

Kết luận & hướng phát triển

Hướng phát triển

Tối ưu hiệu năng:

- Tích hợp tăng tốc bằng GPU/FPGA để giảm thời gian tính toán FHE.

Kết hợp kỹ thuật bảo mật khác:

- FHE + Federated Learning: huấn luyện phân tán, inference an toàn.
- FHE + Differential Privacy: tăng cường bảo vệ chống tấn công suy ngược.

Mở rộng mô hình hỗ trợ:

- Nghiên cứu lượng tử hóa và xấp xỉ hàm phi tuyến hiệu quả hơn cho deep learning trên FHE.

Ứng dụng tiềm năng:

- Y tế: chẩn đoán bệnh từ hình ảnh y khoa mà không lộ dữ liệu bệnh nhân.
- Tài chính: phân tích tín dụng khách hàng trong môi trường zero-trust.
- Chính phủ: xử lý thống kê dân số, an ninh quốc phòng bảo mật cao.

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- ✓ Tổng quan nghiên cứu
- ✓ Phương pháp thực hiện
- ✓ Thực nghiệm
- ✓ Kết luận & hướng phát triển
- **Nguồn tham khảo**



Nguồn tham khảo

- [1] Zama AI. (2023). Presenting Concrete ML at Google Tech Talks. Retrieved from <https://zama.ai>
- [2] Maggio, L. (2024). PPML: Machine Learning on Data You Cannot See. GitHub repository. Retrieved from <https://github.com/leriomaggio/ppml-tutorial>
- [3] Zama AI. (2018). Concrete-ML: Machine Learning with Homomorphic Encryption. GitHub repository. Retrieved from <https://github.com/zama-ai/concrete-ml>
- [4] Vázquez, A. N. (2018). Privacy Preserving Machine Learning. ML4DS Group.
- [5] Fang, H., & Qian, Q. (2021). Privacy preserving machine learning with homomorphic encryption and federated learning. Future Internet, 13(4), 94.
- [6] Lee, J.-W., et al. (2022). Privacy-preserving machine learning with fully homomorphic encryption for deep neural networks. IEEE Access, 10, 30039–30054.

Thank You!



Mã hoá và Thám mã

**Privacy Preserving Machine Learning with
Fully Homomorphic Encryption**

GVHD: TS. Võ Văn Khang

HVTH: Nguyễn Ngọc Đình – 2441862007

Hà Anh Dũng – 2441862005

Nguyễn Minh Trung Nghĩa – 2441861021